M18 # Event-Driven Agents

Methodology Advanced

Triggering agents automatically from GitHub, Linear, Jira

From pull to push

Interactive Claude Code usage is pull-based: you open a terminal, type an instruction, iterate. Event-driven automation removes the human from the triggering step. A card moves to "In Progress" in Linear, an agent starts. A GitHub issue receives the `claude-fix` label, an agent opens a branch within seconds.

The human remains in the loop for final validation (PR review, merge approval), but work initiation happens through the existing project management workflow.

Pipeline architecture

```
Event source
  → Filter (label, mention, state)
    → Context extraction
      → Agent selection
        → Claude Code
          → Result routing (PR, comment, status)
```

Each component is independent and replaceable. The filter determines which events merit an agent; without strict filtering, an event burst can instantiate dozens of agents simultaneously.

Compatible event sources

Source	Triggers	Typical usage
GitHub Issues	Creation, label added	Triage, automatic fix
GitHub PR	Open, comment	Review, corrections
Linear	State change, label	Feature implementation
Jira	Transition, assignment	Technical work, debt
Slack	Message, emoji reaction	Quick investigations
PagerDuty	Incident created	Diagnostic scripts

Activation filter

Do not trigger an agent for every event. A simple bash filter:

```bash
# Trigger only if "claude-auto" label is present
if [[ "$CARD_LABELS" != *"claude-auto"* ]]; then
  echo "Skipping: no claude-auto label"
  exit 0
fi
```

For GitHub, the `@claude review` trigger in a PR comment is the most documented pattern: on-demand triggering without polluting the automated workflow.

Concrete example: Linear agent loop

The most documented pattern (Damian Galarza, February 2026): Linear as single source of truth, Claude Code as implementer.

The ticket description serves as prompt. A ticket with clear acceptance criteria produces quality code; a vague ticket produces vague code, exactly as with a human developer.

```bash
spawn_agent() {
  set -o pipefail
  claude --print --dangerously-skip-permissions \
    "Implement Linear card:
    Title: $title
    Description: $description
    1. Create branch feat/$issue_id
    2. Implement feature, run tests
    3. Open PR" 2>&1 | tee "/tmp/agent-$issue_id.log"
}
```

Mandatory safeguards

Idempotence: check that a branch does not already exist before starting, to avoid double processing in case of a duplicated webhook.

Concurrency limit: set a cap from measured resource use, rate limits and budget. Increase it only when independently completed tasks justify the added coordination.

Circuit breaker: if an agent fails more than 3 times on the same ticket, stop the automation and alert a human.

Human review: agents create PRs, humans approve them. Merge to main must never be automated without validation.

Minimal authentication

The Claude Code token used in CI must be scoped to the minimum necessary. A PR review agent does not need write access to the npm registry or deployment secrets. Separate tokens by agent type and regularly audit their effective permissions.